

Agentic AI in Software Engineering

From Data to Solutions: Weekend 6 – Agentic AI
Piyushi Goyal



Where this lecture sits in the arc



Part 1: From autocomplete to agents

The three generations



2021

Autocomplete

Suggests the next line while you type.

GitHub Copilot



You watch, you accept.



2023

Chat

You ask, it answers, you paste.

ChatGPT · Copilot Chat



You ask, you paste.



2024–2026

Agents

You are “*vibe-coding*” – describe the goal and let the AI handle rest!

Cursor · Claude Code · Devin

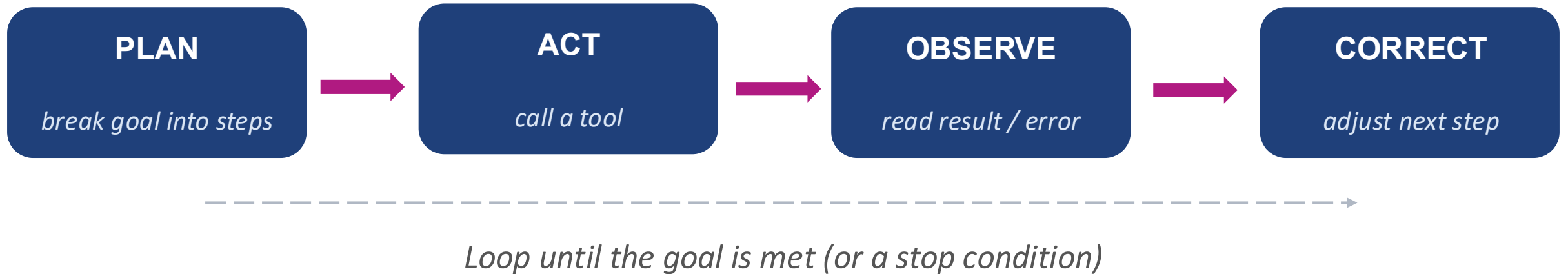


You describe a goal, you review.

ScoutAI lives in the third column.

An agent = LLM + tools + a loop

An agent is a language model given tools and a loop that lets it act, observe, and correct.



Analogy

- A chatbot is **Siri**
- An agent is **JARVIS**: Tony says "run diagnostics on the suit and reorder the palladium", and by the time he lands, it's done.

When to reach for it: workflow vs. agent

WORKFLOW = fixed pipeline



- Use when steps are known in advance.
- Cheaper, more predictable, easier to test.

AGENT = steps chosen at runtime



- Use when the model must choose its next step based on what it just saw.
- More powerful, harder to audit, costs more per run.

Rule of thumb: if you can draw the pipeline on a whiteboard, build a workflow. Only reach for an agent when you can't.

Part 2: Building ScoutAI

Meet ScoutAI — the app we'll build together

"Give me a one-page brief on Competitor X."

The old way

2 days of a junior analyst's time.

- Read the annual financial reports
- Scrape the website for pricing
- Check LinkedIn for hiring signals
- Pull the latest news headlines
- Write the memo

With ScoutAI

~4 minutes.

- Type: analyse("Competitor X")
- Agent plans, fetches, drafts
- Live prices from yfinance
- Real filings from SEC EDGAR
- Human reviews and signs off

Under the hood — where the LLM comes from



OpenRouter

one endpoint,
many models



HuggingFace

open-weights,
self-host



Anthropic, OpenAI, Gemini

API-based frontier models



**AWS Bedrock /
Azure Foundry /
Google Vertex AI**

cloud-native gateways

Start here: a plain LLM hits a wall

**"One-page brief on NVIDIA.
Live price, last week's news,
risk factors."**

A chatbot alone cannot do this.
Here's why →

Five gaps a plain LLM leaves



Stale price

training data ends months ago



Hallucinated news

confident, plausible, wrong



No sources

you can't verify a single claim



Drifting format

each run looks different



No memory

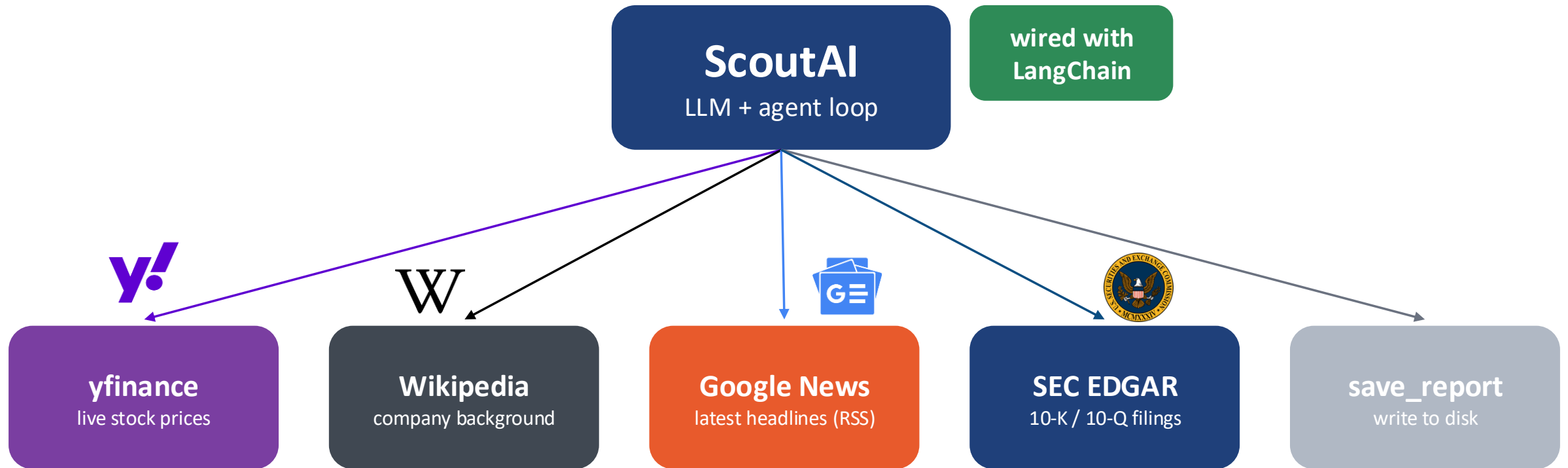
yesterday's notes vanish

Tools: the agent's hands



A tool = any function the LLM is allowed to call.

It can be an API call, a shell command, a database query, a calculator; anything that lets the model act beyond text.



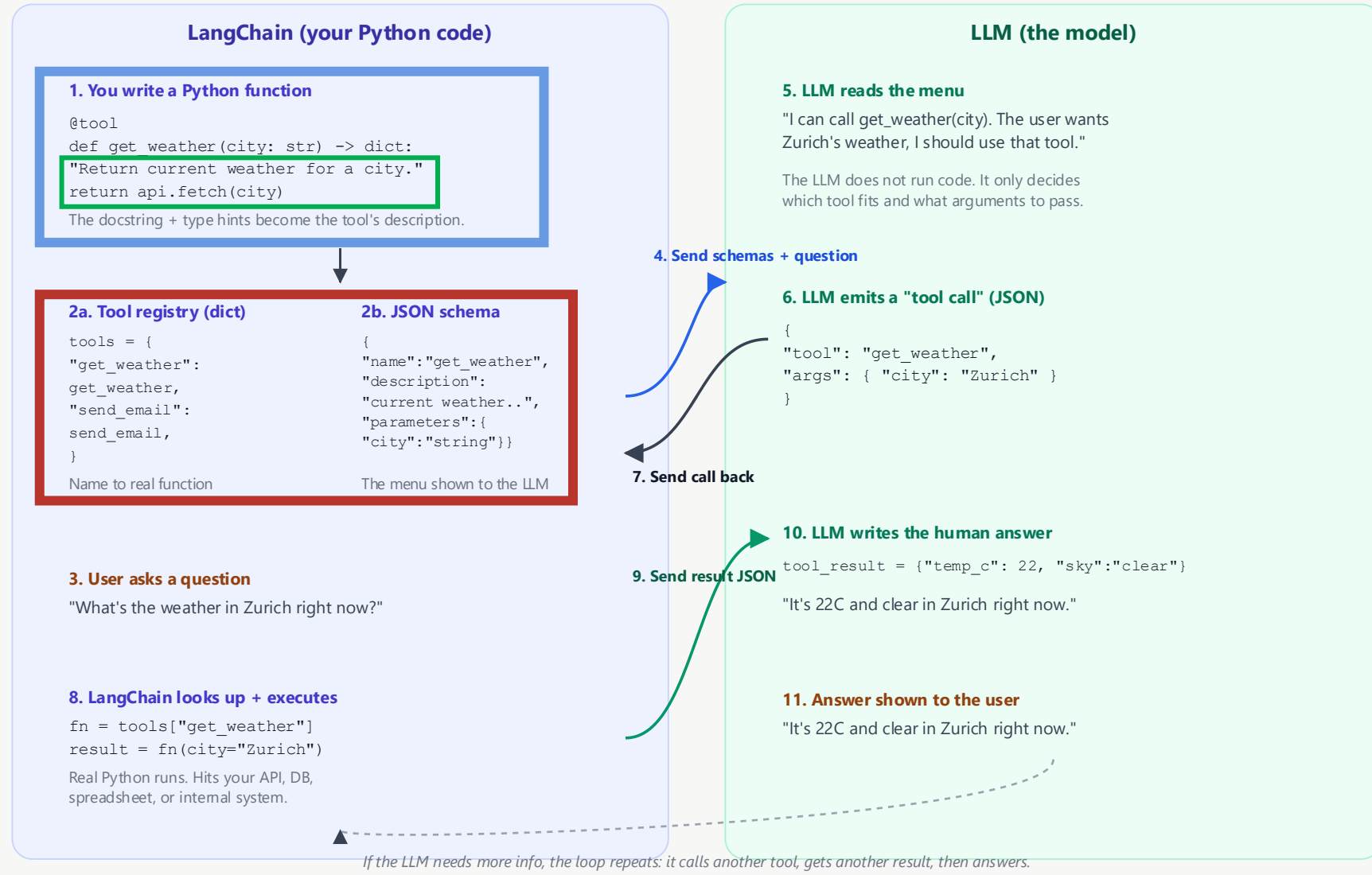
In the CX, every one of these five hits a real, live API.

Tools: LangChain and LLM

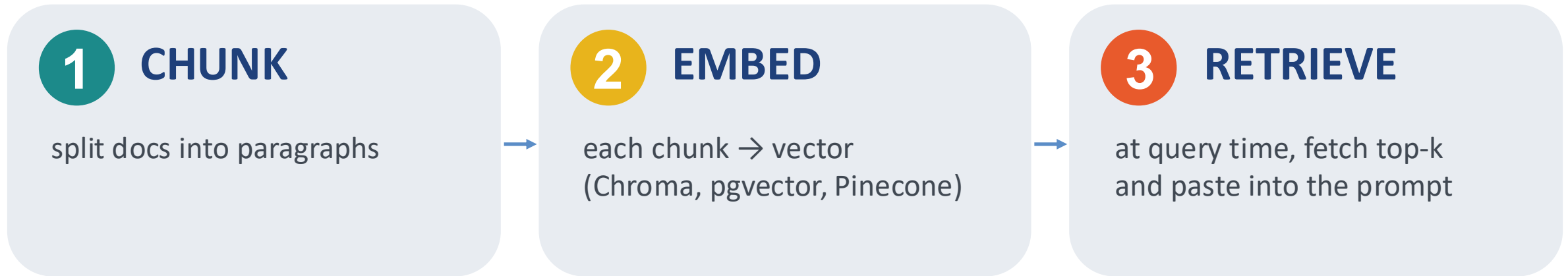


How LangChain and an LLM call a tool

A round trip: Python function to schema to LLM decision to execution to human answer



RAG: giving the agent your private knowledge



In ScoutAI:

Five internal memos (channel checks + house rules) are chunked, embedded in Chroma, and exposed to the agent as a `search_our_research` tool.

The agent decides on its own when to call it — no if-statement.

RAG = unstructured knowledge (docs). Tools = live data and actions. Use both.

Skills: reusable playbooks the agent invokes

What Skills look like

A Skill directory:

```
anthropic_brand/  
├ SKILL.md  
├ docs.md  
├ slide-decks.md  
└ apply_template.py
```

anthropic/brand_style/SKILL.md

YAML Fontmatter

```
---  
name: Anthropic Brand Style Guidelines  
description: Applies Anthropic's official brand colors and  
typography to PowerPoint presentations  
---
```

Metadata

Markdown

Overview

This skill provides Anthropic's official brand identity resources for PowerPoint presentations. It includes a pre-branded template and tools to apply Anthropic styling to existing presentations.

Freeform Content

Colors

Main Colors:

- Dark: `#141413` - Primary text and dark backgrounds
- Light: `#faf9f5` - Light backgrounds and text on dark
- Light Gray: `#e8e6dc` - Subtle backgrounds

Typography

- *Headings*: Poppins (with Arial fallback)
- *Body Text*: Lora (with Georgia fallback)

Without

same prompt → different-looking report each run.

With

identical structure. Auditable, reviewable, versionable.

2025-26

Anthropic, Cursor, OpenAI all ship Skills as first-class.

Skills: reusable playbooks the agent invokes

Bundling additional content

pdf/SKILL.md

YAML Frontmatter

```
---
name: pdf
description: Comprehensive PDF toolkit for extracting text and
tables, merging/splitting documents, and filling-out forms.
---
```

Markdown

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see `./reference.md`. If you need to fill out a PDF form, read `./forms.md` and follow its instructions.

Quick Start

```
```python
from pypdf import PdfReader, PdfWriter
```

#### # Read a PDF

```
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

#### # Extract text

```
text = ""
for page in reader.pages:
 text += page.extract_text()
```

pdf/reference.md

### # PDF Processing Advanced Reference

This document contains advanced PDF processing features, detailed examples, and additional libraries not covered in the main skill instructions.

### ## pypdfium2 Library (Apache/BSD License)

#### ### Overview

pypdfium2 is a Python binding for PDFium (Chromium's PDF library). It's excellent for fast PDF rendering, image generation, and serves as ...

pdf/forms.md

If you need to fill out a PDF form, first check to see if the PDF has fillable form fields. Run this script from this file's directory:

```
`python scripts/check_fillable_fields <file.pdf>`,
and depending on the result go to either the "Fillable
fields" or "Non-fillable fields" and follow those
instructions.
```

#### # Fillable fields

If the PDF has fillable form fields:

- Run this script from this file's directory:

```
`python scripts/extract_form_field_info.py <input.pdf>
<fields.json>`.
...
```

# Skills: reusable playbooks the agent invokes

## Skills and the Context Window



# Memory: what the agent remembers

## SHORT-TERM · context window

USER

Analyse Competitor X

SCOUT

Reading 10-K...

SCOUT

Q3 revenue: \$2.1B

SCOUT

Retrieved 4 quotes on margin

SCOUT

Drafting section 2 of 5...

✗ Gone when the session ends.

## LONG-TERM · persistent memory

*Vector DB / KV store · Mem0, LangMem, Zep, Memory.md*

House style: prefer bullets.

We covered Comp X in June — reuse.

CFO wants risk section first.

Never quote unpublished figures.

Tone: cautious, hedged.

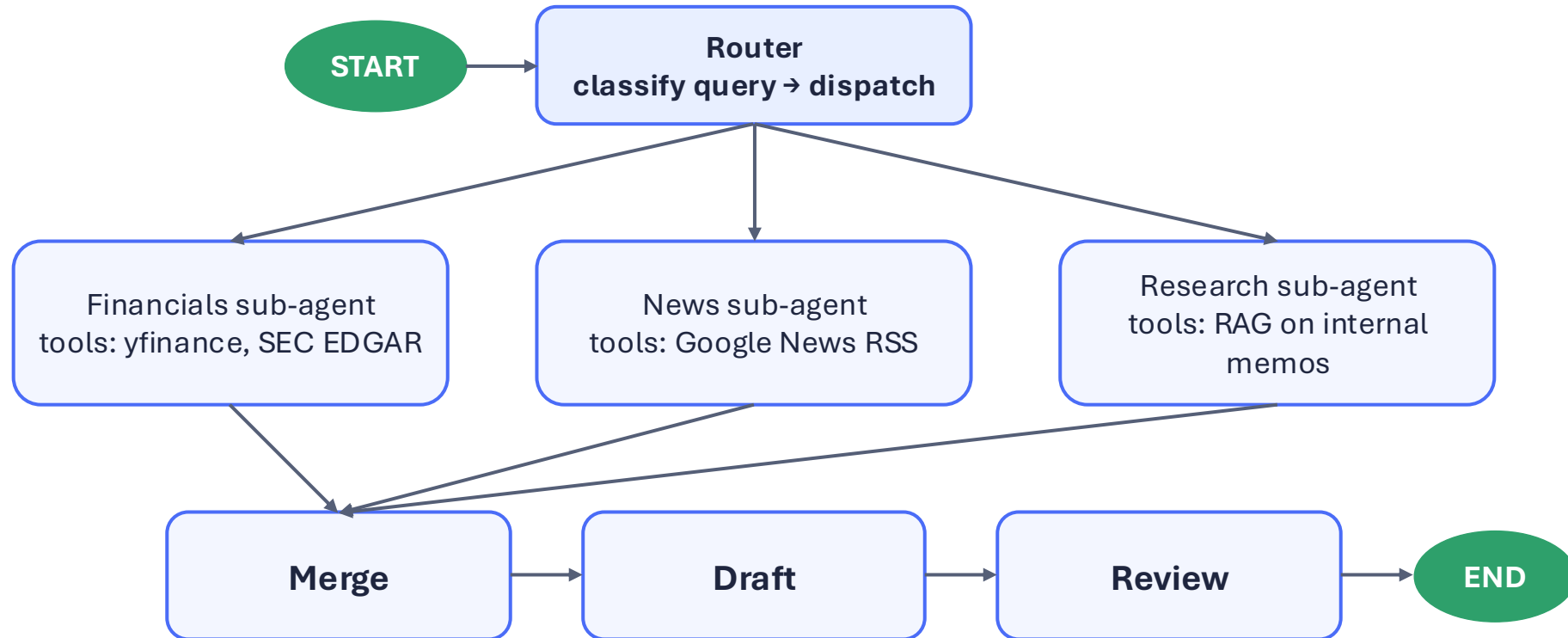
✓ Survives across sessions and across agents.



# LangGraph: an orchestrator that dispatches sub-agents



*The orchestrator doesn't do the work. It routes to specialist sub-agents, each with its own tools and system prompt.*



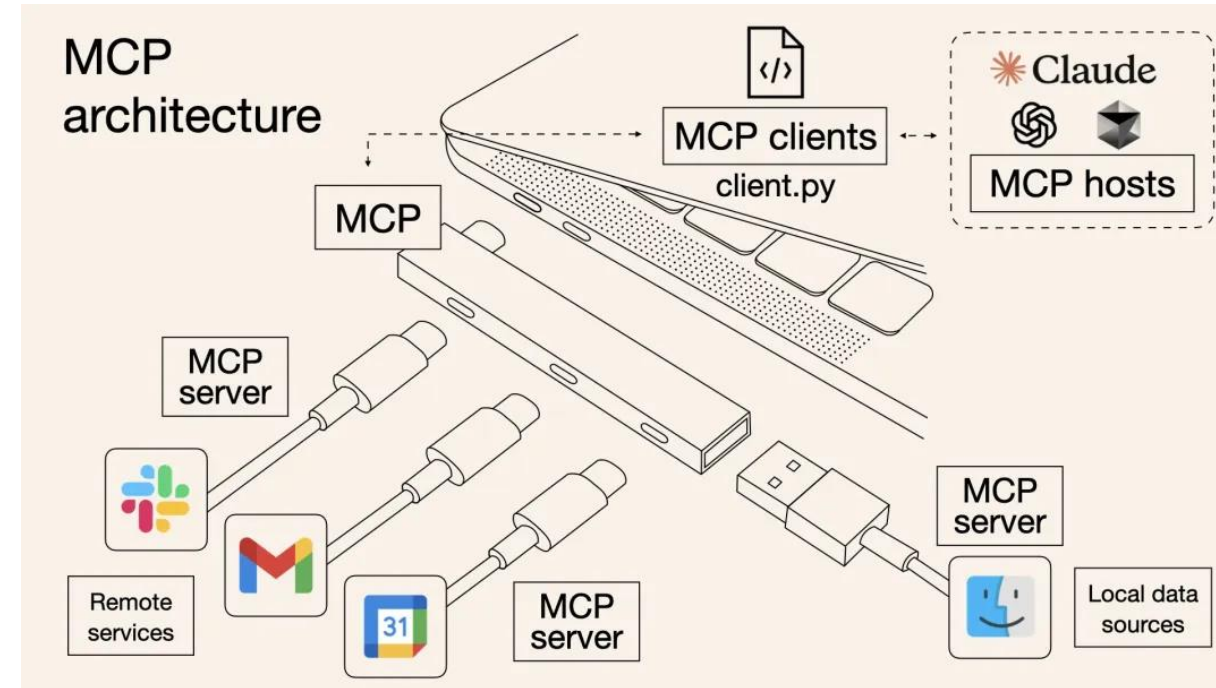
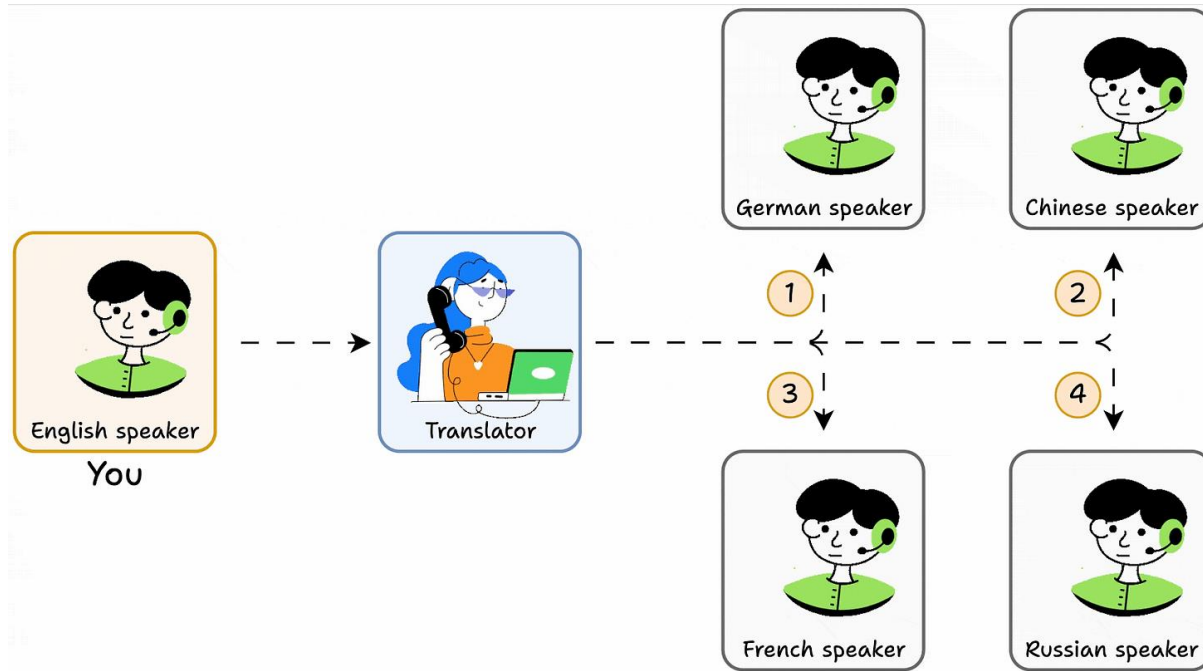
Uber



cisco

*Reach for it whenever the workflow needs branches, loops, or multiple agents; draw the graph first, then wire the nodes.*

# Model Context Protocol (MCP): acts like a USB-C



**Open-sourced by Anthropic, Nov 2024. Backed by OpenAI, Google, Microsoft.**  
*The single most important protocol name to know. Now backed by Anthropic, OpenAI, Google, Microsoft.*

# MCP architecture: host · client · server

## HOST

user-facing app (Claude Desktop, Cursor, ChatGPT, your agent)

## CLIENT

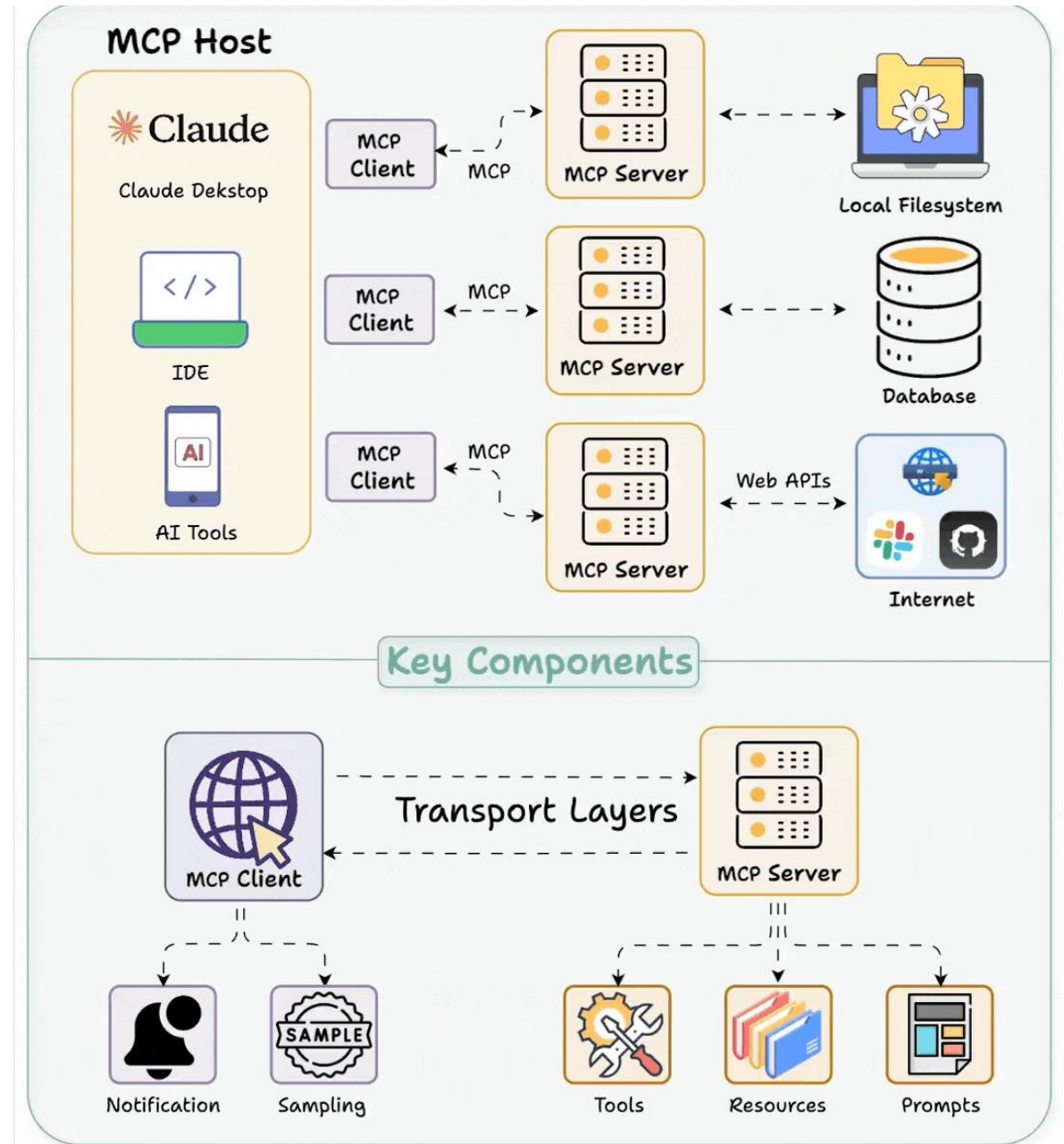
speaks MCP; lives inside the host (one per open server)

## SERVER

wraps an app or data source (GitHub, Postgres, Slack, your API)

stdio · local process, no network hop  
(filesystem, terminal, local apps)

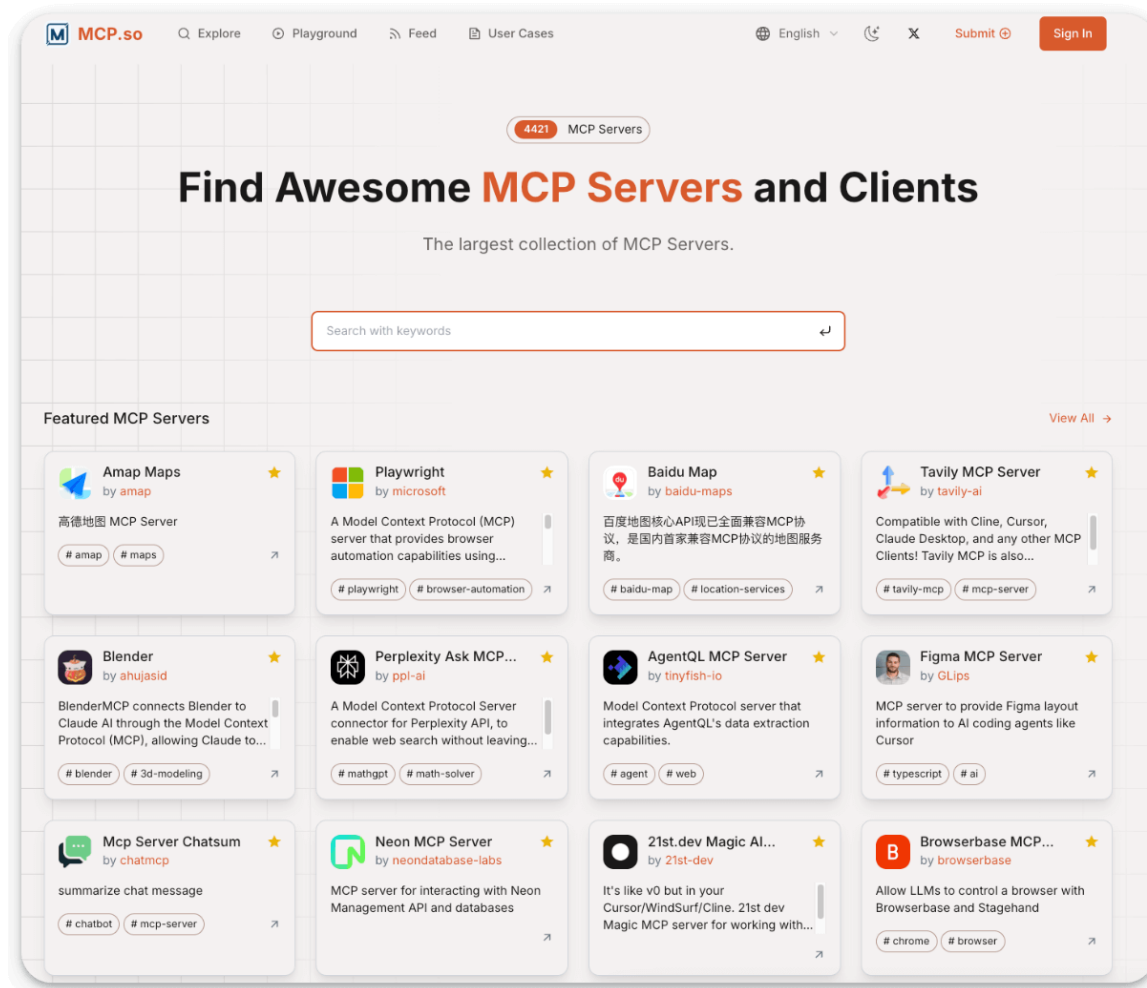
HTTP / SSE · remote server, org-wide (Slack, Github, Notion, Salesforce)



# Where servers and clients come from

## 1. MCP Registry

whole apps as tool bundles



## 2. PyPI / npm

traditional package registries

`pip install langchain-slack`

`npm install @langchain/openai`

500+ LangChain connectors

## 3. Your own

company-internal tools

`query_our_datawarehouse`

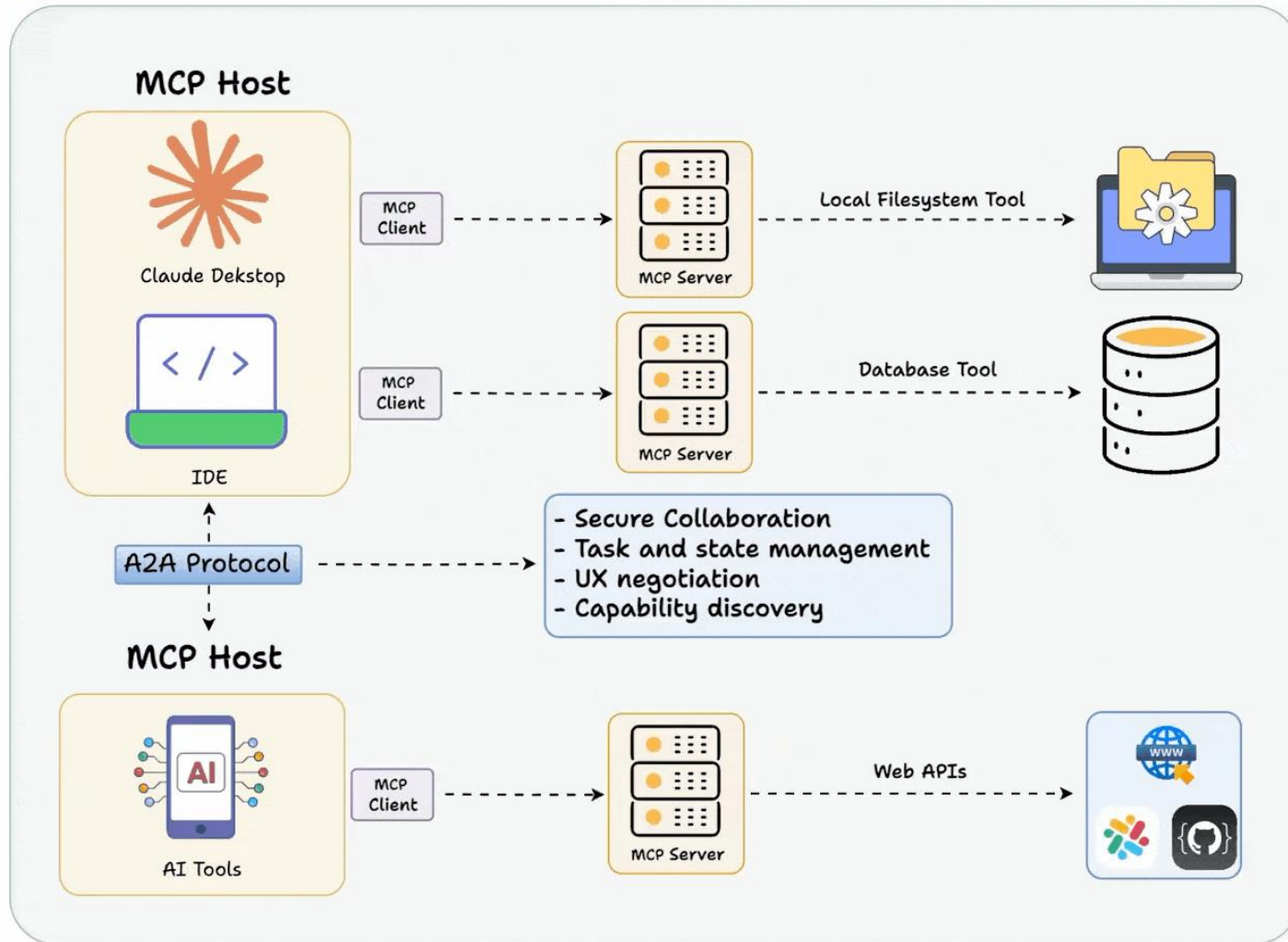
`post_to_our Pagerduty`

`check_our_compliance_rules`



# Agent to Agent (A2A) protocol: agents talking to agents

## MCP vs. A2A protocol DailyDoseOfDS.com



 **Think Nick Fury** 

*Fury doesn't fight aliens himself. He calls the Avengers.*

Orchestrator agent = Fury.  
Specialist agents = Avengers.

MCP = each hero's gear. A2A = the comms earpiece.

# A2A: agents talking to agents



# A2A in the wild

## Google Cloud: hiring workflow

Hiring manager's agent → sourcing agent → background-check agent. Three agents, three vendors, one job.

[Link to the demo!](#)

## Salesforce Agentforce ↔ Google Agentspace

A Salesforce service agent delegates to a Google-side agent for Workspace context. Zero custom integration.

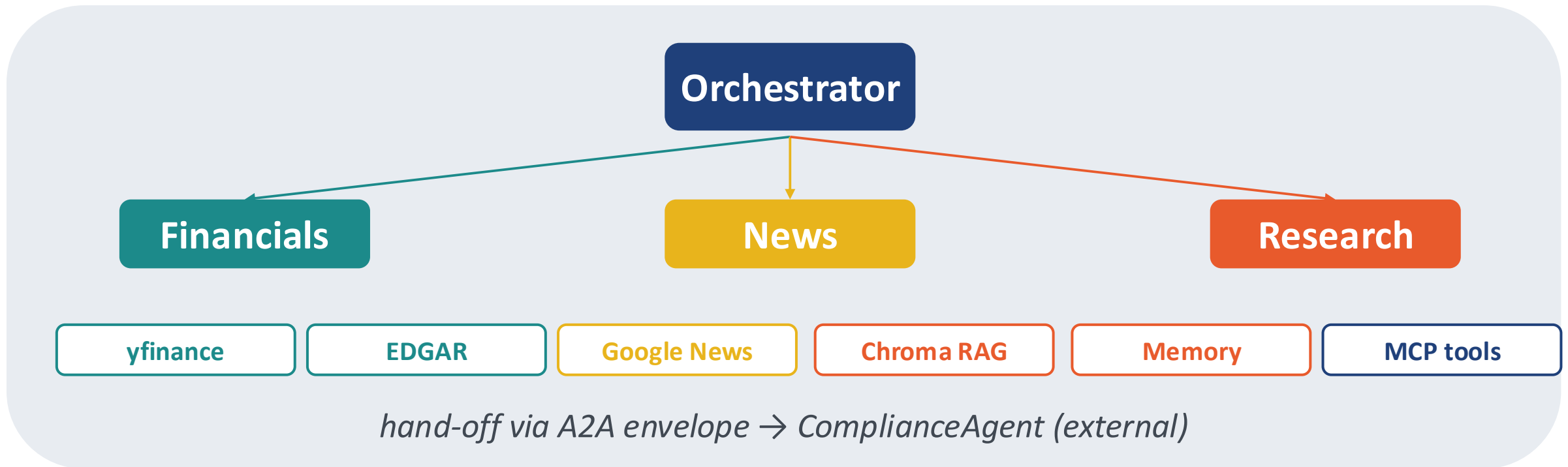
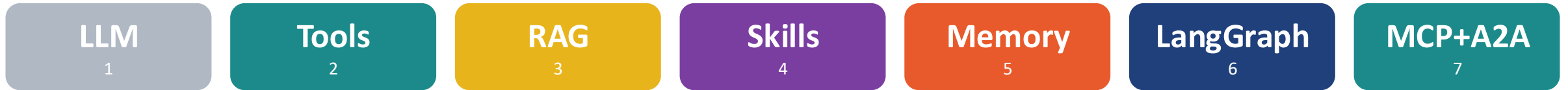
## SAP Joule & ServiceNow Now Assist

Both platforms adopted A2A so their agents interoperate outside their own walls.

## Enterprise pilots: Deloitte, Accenture, BCG

Coordinating agents across their internal stack and their clients' stacks.

# ScoutAI, fully assembled



*The CX builds this from left to right, one concept per part.*



## Part 3: Agents across the SDLC

# The whole lifecycle

*Same seven primitives from Part 2. Different stage, same machine.*

Stage 1	Stage 2	Stage 3	Stage 4	Stage 5	Stage 6
<b>Reqs &amp; Design</b>  Draft user stories, PRDs, acceptance criteria  <b>TOOLS YOU'LL HEAR</b>  Notion AI · ChatGPT ReadAI  <b>⚠ WATCH OUT</b>  invented constraints	<b>Coding</b>  Multi-file edits, refactors, feature implementation  <b>TOOLS YOU'LL HEAR</b>  Cursor · Claude Code GPT Codex · Copilot · Devin  <b>⚠ WATCH OUT</b>  shipped ≠ value	<b>Code Review</b>  First-pass PR review, diff summaries, security flags  <b>TOOLS YOU'LL HEAR</b>  GitHub Copilot Review CodeRabbit  <b>⚠ WATCH OUT</b>  amplifies weak culture	<b>Testing &amp; QA</b>  Unit tests, E2E flows, edge-case fuzzing  <b>TOOLS YOU'LL HEAR</b>  Meticulous · Momentic Playwright + Claude  <b>⚠ WATCH OUT</b>  both sides of assertion	<b>Debug &amp; Incidents</b>  Log triage, hypotheses, hotfixes, post- mortems  <b>TOOLS YOU'LL HEAR</b>  Sentry AI · Cursor 'fix' Runbook agents  <b>⚠ WATCH OUT</b>  confidently wrong	<b>DevOps &amp; Deploy</b>  Terraform / Helm / CI, dependency bumps  <b>TOOLS YOU'LL HEAR</b>  Dependabot · Renovate Claude Code w/ shell  <b>⚠ WATCH OUT</b>  kubectl + bad idea

# Zoom in: Code Review

*The stage where the ROI conversation actually lives.*

## The argument

- 1** When agents write more code, review becomes the bottleneck.
- 2** Review is where quality, security, and standards enter your codebase.
- 3** Hence, a high-priority agent is the reviewer, not the coder.

## Capacity shift

Writing capacity

**3× with agents**

Review capacity

flat

**→ the new bottleneck**

# The PR-review agent: same seven pieces

*Compare pill-for-pill with slide 22. Different application. Same primitives.*

1 LLM	2 Tools	3 RAG	4 Skills	5 Memory	6 LangGraph	7 MCP + A2A
the reasoner reading the diff	git diff · run tests static analysis · coverage	your codebase past PRs · ADRs	review checklist as versioned playbook	team preferences (who hates what)	sub-agents: security · style · tests · performance	MCP → GitHub / CI A2A → ComplianceA gent

diff → plan → call tools → observe → post review comments → hand off to ComplianceAgent

# One matrix: same primitives, every stage

Tool / concept	Reqs	Code	Review	Test	Debug	DevOps
Cursor / Claude Code	●	●	●	●	●	—
MCP servers	●	●	●	●	●	●
RAG (internal docs)	●	—	●	—	●	—
Skills	●	●	●	●	●	—
LangGraph flows	●	—	●	●	●	—
Sub-agents / A2A	●	●	●	●	●	—

*The 'Cursor vs Copilot vs Claude Code' argument matters less than which primitives your team is fluent in.*

## Part 4: Where humans stay indispensable

# Three things agents remain bad at

**01**

## Deciding what to build

An agent will happily build the wrong thing beautifully. Prioritisation is a judgment call about people and money.

**02**

## Owning a decision

When it's wrong, nobody's fired. Accountability doesn't survive being handed to a model.

**03**

## Novel taste

Agents interpolate from training data. Anything genuinely new — a brand voice, a bold architecture — is on you.

*The engineers who thrive shift from typing code to reviewing, testing, and directing agents.*



# Human-in-the-loop: pick a pattern

## HIGH RISK

### Approve-to-act

Agent proposes. Human clicks yes.

#### USE WHEN

*Anything that costs money, sends email, or writes to prod.*

## BULK WORK

### Sample-and-audit

Agent acts freely. Human reviews N% of outputs.

#### USE WHEN

*High-volume, low-stakes work.*

## MIXED

### Escalate-on-uncertainty

Agent handles the easy 80%. Flags the hard 20%.

#### USE WHEN

*Support, triage, classification.*

*The guardrail is you. Pick the pattern per workflow; don't default.*

**The leverage isn't writing code faster.**

*It is redesigning the work to move humans off repetitive tasks and onto the judgment, taste, and accountability that agents still can't fake.*

Thank you!